

Skill-Centric Unified Job Search Aggregator System — Engineering Rules & Guardrails

These rules apply to anyone (or any agent) contributing code to this project. They exist to protect the two things the synopsis identifies as load-bearing: **data compliance/integrity** and **relevance quality**. Rules are grouped by concern; each is a hard rule unless marked "guideline."

1. Data Source Compliance Rules

Every scraper module **must** check and respect the target platform's `robots.txt` before any request is issued.

No ingestion source is added without a documented rate-limit policy (token-bucket config) checked into the source's module.

On `429 / 503` responses, workers **must** back off exponentially with jitter — never retry immediately, never in a tight loop.

Proxy rotation, where used, is a resilience mechanism only. It must never be configured or documented as a rate-limit bypass. Any PR that frames proxy use this way is rejected.

Scraper-based ingestion is only added for platforms with no usable official/partner API. If a source later ships an API, migrate to it.

Ingestion windows for different sources must be staggered in the scheduler config, not left to run concurrently by default.

2. Database & Schema Rules

All schema changes go through migrations (e.g. Alembic) — no direct DDL against staging or production.

The `jobs.UNIQUE (source_platform, source_job_id)` and `dedup_hash` index are load-bearing for concurrency safety — never removed or made nullable without an RFC.

Junction tables (`user_skills` , `job_skills` , `applications`) hold relationship-specific attributes; new relationship-scoped fields (e.g. a future "endorsement count") go on the junction table, never bolted onto `users` or `skills` directly.

`relevance_score` and `match_score` are constrained to `[0, 1]` at the DB layer (`CHECK` constraints) — application code must not assume it can write outside that range and rely on the DB to catch it; validate before insert too.

No skill string is ever written directly to `user_skills / job_skills` without resolving to a canonical `skills.skill_id` first — this is what prevents "JavaScript" vs "javascript" vs "Javascript" fragmentation.

3. NLP & Matching Rules

Phase A (extraction) runs asynchronously. No code path may call the spaCy extraction pipeline synchronously inside a request handler.

Any change to the gazetteer or NER model must be validated against the annotated test sample (from `phase.md` Sprint 3) before merge, with precision/recall reported in the PR.

Any change to deduplication thresholds (company ≥ 0.90 , title ≥ 0.85 Levenshtein/Jaccard) must be validated against the labeled duplicate/non-duplicate test set (Sprint 2) before merge.

Match scores must always ship with an explanation (overlapping/missing skills) in the API response — a bare numeric score is not an acceptable `/feed` response shape.

```
match_score ∈ [0,1] — context and the existing job_skills, user_skills storage schema.
```

4. API Design Rules

All endpoints are versioned under `/api/v1/...`. Breaking changes get a new version, not an in-place change.

Every endpoint must be covered by FastAPI's generated OpenAPI schema — no undocumented routes.

`GET /feed` filters (location, employment_type, is_remote, min_match_score) compose with ranking; they must never silently override or disable the semantic ranking.

Error responses follow a consistent shape (status code + machine-readable error code + human message) across all endpoints — no ad-hoc error formats per route.

5. Security & Secrets Rules

Passwords are hashed (never stored plaintext) before hitting `users.password_hash`.

No secrets (API keys, DB credentials, proxy credentials) are committed to the repo. They live in environment variables / a secrets manager, referenced via `.env.example` placeholders only.

PII (email, location fields) is not logged in plaintext in application logs.

Auth tokens/session handling must be validated on every protected route — no route assumes prior middleware ran without an explicit dependency declaration (FastAPI `Depends`).

6. Code & Workflow Rules (guidelines unless noted)

Python backend/worker code: PEP 8, type hints on public functions, formatted with a consistent auto-formatter (e.g. `black`).

Frontend: consistent component structure per `design.md §5`; no inline styles for anything beyond one-off layout tweaks.

Git: feature branches per phase/task, descriptive commit messages, PRs reference the relevant phase from `phase.md`.

Hard rule: no PR merges into a phase's integration point without meeting that phase's exit criteria as defined in `phase.md`.

Hard rule: scope changes (adding something from the "Out of Scope" list in the synopsis, e.g. multi-lingual parsing, native mobile, employer-facing tooling) require an explicit decision logged in `memory.md` before work starts — no silent scope creep.

7. Testing Rules

Deduplication and NLP extraction changes require regression tests against their respective labeled test sets (see §2 and §3 above) — this is non-negotiable, not a nice-to-have.

New API endpoints ship with at least one happy-path and one failure-path test.

Schema migrations are tested against a seeded copy of the staging environment before being applied to production.

Companion documents: `Architecture.md` (system structure), `design.md` (schema/API/algorithms), `phase.md` (delivery plan), `memory.md` (decision log).